

📦📦📦📦📦 📦📦📦📦📦

📦📦📦📦📦 📦📦📦📦📦📦 📦📦📦📦📦📦📦

📦📦📦📦 📦📦 📦📦📦📦📦 📦📦📦📦 📦📦📦📦📦📦
📦📦📦📦 📦📦📦📦📦📦 📦📦📦📦📦 📦📦📦📦📦📦 📦📦📦📦📦📦

Next.js 15 + Prisma + SQLite

📁📁📁📁 📁📁📁📁📁📁

1. package.json - 📁📁📁📁📁📁 📁📁📁📁📁📁 📁📁📁📁📁📁📁📁
2. next.config.ts - 📁📁📁📁📁📁 Next.js
3. tsconfig.json - 📁📁📁📁📁📁 TypeScript
4. prisma/schema.prisma - 📁📁📁📁📁 📁📁📁📁📁 📁📁📁📁📁📁📁📁
5. src/lib/prisma.ts - 📁📁📁📁📁 📁📁📁📁📁 📁📁📁📁📁📁📁📁
6. src/app/layout.tsx - 📁📁📁📁📁📁📁 📁📁📁📁📁📁
7. src/app/globals.css - 📁📁📁📁📁📁📁 📁📁📁📁📁📁
8. src/app/api/auth/register/route.ts - API 📁📁📁📁📁 📁📁📁📁
9. src/app/api/auth/login/route.ts - API 📁📁📁📁📁 📁📁📁📁
10. src/app/api/auth/logout/route.ts - API 📁📁📁📁📁 📁📁📁📁
11. src/app/api/auth/me/route.ts - API 📁📁📁📁📁📁📁📁 📁📁📁📁📁📁
12. src/app/api/sync/route.ts - API 📁📁📁📁📁📁 📁📁📁📁📁📁📁📁
13. src/app/api/suggestions/delete/route.ts - API 📁📁📁 📁📁📁📁📁📁📁📁📁📁📁
14. src/app/api/classify/route.ts - API 📁📁📁📁📁 📁📁📁📁📁📁📁📁
15. src/app/api/family/route.ts - API 📁📁📁📁📁📁📁
16. src/app/api/invite/route.ts - API 📁📁📁📁📁📁📁
17. src/app/api/scan-receipt/route.ts - API 📁📁📁 📁📁📁📁📁📁📁📁
18. public/manifest.json - 📁📁📁 PWA
19. public/sw.js - Service Worker
20. src/app/page.tsx - 📁📁📁📁📁📁 📁📁📁📁📁📁📁📁 (3000+ 📁📁📁)

1. package.json

□ /home/z/my-project/package.json

```
{
  "name": "home-items-manager",
  "version": "1.0.0",
  "private": true,
  "scripts": {
    "dev": "next dev -p 3000",
    "build": "prisma generate && next build",
    "start": "next start -p 3000",
    "lint": "eslint .",
    "db:generate": "prisma generate",
    "db:push": "prisma db push",
    "db:migrate": "prisma migrate dev"
  },
  "dependencies": {
    "@prisma/client": "5",
    "@radix-ui/react-dialog": "^1.1.14",
    "@radix-ui/react-progress": "^1.1.7",
    "class-variance-authority": "^0.7.1",
    "clsx": "^2.1.1",
    "dotenv": "^17.3.1",
    "lucide-react": "^0.525.0",
    "next": "^16.1.1",
    "prisma": "5",
    "react": "^19.0.0",
    "react-dom": "^19.0.0",
    "tailwind-merge": "^3.3.1",
    "tailwindcss-animate": "^1.0.7",
    "z-ai-web-dev-sdk": "^0.0.16"
  },
  "devDependencies": {
    "@tailwindcss/postcss": "^4",
    "@types/node": "25.2.3",
    "@types/react": "^19",
    "@types/react-dom": "^19",
    "eslint": "^9",
    "eslint-config-next": "^16.1.1",
    "tailwindcss": "^4",
    "tw-animate-css": "^1.3.5",
    "typescript": "^5"
  }
}
```

2. next.config.ts

□ /home/z/my-project/next.config.ts

```
import type { NextConfig } from "next";

const nextConfig: NextConfig = {
  typescript: {
    ignoreBuildErrors: true,
  },
  reactStrictMode: false,
};

export default nextConfig;
```

3. tsconfig.json

□ /home/z/my-project/tsconfig.json

```
{
  "compilerOptions": {
    "target": "ES2017",
    "lib": [
      "dom",
      "dom.iterable",
      "esnext"
    ],
    "allowJs": true,
    "skipLibCheck": true,
    "strict": true,
    "noEmit": true,
    "noImplicitAny": false,
    "esModuleInterop": true,
    "module": "esnext",
    "moduleResolution": "bundler",
    "resolveJsonModule": true,
    "isolatedModules": true,
    "jsx": "react-jsx",
    "incremental": true,
    "plugins": [
      {
        "name": "next"
      }
    ],
    "paths": {
      "@/*": [
        "./src/*"
      ]
    }
  },
  "include": [
    "next-env.d.ts",
    "**/*.ts",
    "**/*.tsx",
    ".next/types/**/*.ts",
    ".next/dev/types/**/*.ts"
  ],
  "exclude": [
    "node_modules"
  ]
}
```

4. prisma/schema.prisma

```
❏ /home/z/my-project/prisma/schema.prisma
```

```
generator client {
  provider = "prisma-client-js"
}

datasource db {
  provider = "sqlite"
  url      = env("DATABASE_URL")
}

model User {
  id          String      @id @default(cuid())
  name        String
  email       String      @unique
  password    String
  avatar      String      @default("")
  createdAt   DateTime     @default(now())
  updatedAt   DateTime     @updatedAt

  // وضع وأهلاً عيشة نوكي نأ نكمي مدمتسملا - أهلاً لآ
  familyId    String?
  family      Family?      @relation(fields: [familyId], references: [id], onDelete: SetNull)
  familyRole  String      @default("member") // owner, admin, member

  budget      Budget?
  customStores CustomStore[]
  customCategories CustomCategory[]
  familyMembers FamilyMember[]
  items       Item[]
  priceHistory PriceHistoryRecord[]
  sessions    Session[]
  sentInvitations FamilyInvitation[] @relation("SentInvitations")
  receivedInvitations FamilyInvitation[] @relation("ReceivedInvitations")
  savedProductNames SavedProductName[]
}

model Family {
  id          String      @id @default(cuid())
  name        String
  inviteCode   String      @unique
  createdAt    DateTime   @default(now())
  updatedAt    DateTime   @updatedAt

  members      User[]
  items        Item[]
  customStores CustomStore[]
  customCategories CustomCategory[]
  priceHistory PriceHistoryRecord[]
  budget       Budget?
  familyMembers FamilyMember[]
  invitations  FamilyInvitation[]
}

model FamilyInvitation {
  id          String      @id @default(cuid())
  code        String      @unique
  email       String
  status      String      @default("pending") // pending, accepted, rejected
  role        String      @default("member") // admin, member
  createdAt   DateTime     @default(now())
  expiresAt   DateTime

  familyId    String
  family      Family      @relation(fields: [familyId], references: [id], onDelete: Cascade)
  invitedBy   String
  inviter     User        @relation("SentInvitations", fields: [invitedBy], references: [id], onDelete: Cascade)
  invitedUser User?       @relation("ReceivedInvitations", fields: [invitedUserId], references: [id], onDelete: SetNull)
  invitedUserId String?

  @@index([familyId])
  @@index([code])
}

model Session {
```

```

    id      String   @id @default(cuid())
    userId   String
    createdAt DateTime @default(now())
    expiresAt DateTime
    user      User    @relation(fields: [userId], references: [id], onDelete: Cascade)

    @@index([userId])
}

model Item {
  id      String   @id @default(cuid())
  name     String
  category String
  quantity Int      @default(1)
  notes    String   @default("")
  isPurchased Boolean @default(false)
  image    String?
  priority Int      @default(0)
  order    Int      @default(0)
  createdAt DateTime @default(now())
  createdBy String   @default("")
  userId   String
  user      User    @relation(fields: [userId], references: [id], onDelete: Cascade)
  familyId String?
  family    Family? @relation(fields: [familyId], references: [id], onDelete: Cascade)
  prices    Price[]

  @@index([userId])
  @@index([familyId])
}

model Price {
  id      String   @id @default(cuid())
  store   String
  price   Float
  date    DateTime @default(now())
  itemId  String
  item    Item     @relation(fields: [itemId], references: [id], onDelete: Cascade)

  @@index([itemId])
}

model FamilyMember {
  id      String @id @default(cuid())
  name    String
  avatar  String @default("")
  userId  String
  user    User   @relation(fields: [userId], references: [id], onDelete: Cascade)
  familyId String?
  family  Family? @relation(fields: [familyId], references: [id], onDelete: Cascade)

  @@index([userId])
  @@index([familyId])
}

model CustomStore {
  id      String @id @default(cuid())
  name    String
  userId  String
  user    User   @relation(fields: [userId], references: [id], onDelete: Cascade)
  familyId String?
  family  Family? @relation(fields: [familyId], references: [id], onDelete: Cascade)

  @@index([userId])
  @@index([familyId])
}

model PriceHistoryRecord {
  id          String   @id @default(cuid())
  productName String
  store       String
  price       Float
  date        DateTime @default(now())
  userId      String
  user        User     @relation(fields: [userId], references: [id], onDelete: Cascade)
  familyId    String?
  family      Family?  @relation(fields: [familyId], references: [id], onDelete: Cascade)

  @@index([userId, productName])
}

```

```

    @@index([familyId])
}

model Budget {
  id          String    @id @default(cuid())
  monthlyBudget Float    @default(0)
  spentAmount Float     @default(0)
  startDate   DateTime?
  shoppingTurn String?
  userId       String?   @unique
  user         User?     @relation(fields: [userId], references: [id], onDelete: Cascade)
  familyId     String?   @unique
  family       Family?   @relation(fields: [familyId], references: [id], onDelete: Cascade)
}

model CustomCategory {
  id          String    @id @default(cuid())
  name        String
  icon        String    @default("")
  color       String    @default("gray")
  keywords    String    @default("")
  userId       String
  user         User     @relation(fields: [userId], references: [id], onDelete: Cascade)
  familyId     String?
  family       Family?  @relation(fields: [familyId], references: [id], onDelete: Cascade)

  @@index([userId])
  @@index([familyId])
}

// تاج ارتقا للة طوف حمله تاج تملء اسماء
model SavedProductName {
  id          String    @id @default(cuid())
  name        String
  userId       String
  user         User     @relation(fields: [userId], references: [id], onDelete: Cascade)
  createdAt   DateTime @default(now())

  @@unique([userId, name])
  @@index([userId])
}

```


5. src/lib/prisma.ts

□ /home/z/my-project/src/lib/prisma.ts

```
import { PrismaClient } from '@prisma/client'

const globalForPrisma = globalThis as unknown as {
  prisma: PrismaClient | undefined
}

export const prisma = globalForPrisma.prisma ?? new PrismaClient()

if (process.env.NODE_ENV !== 'production') globalForPrisma.prisma = prisma
```

6. src/app/layout.tsx

```
□ /home/z/my-project/src/app/layout.tsx
```

```
import type { Metadatas, Viewport } from "next";
import { Geist, Geist_Mono } from "next/font/google";
import "./globals.css";

const geistSans = Geist({
  variable: "--font-geist-sans",
  subsets: ["latin"],
});

const geistMono = Geist_Mono({
  variable: "--font-geist-mono",
  subsets: ["latin"],
});

export const metadata: Metadata = {
  title: "علاء كذب تايرت شهمل - ميظنت - يضا قم",
  description: "علاء كذب تايرت شهمل - ميظنت - يضا قم كي كذا يضا قم قيبط",
  manifest: "/manifest.json",
  icons: {
    icon: [
      { url: "/icons/icon-72x72.png", sizes: "72x72", type: "image/png" },
      { url: "/icons/icon-96x96.png", sizes: "96x96", type: "image/png" },
      { url: "/icons/icon-128x128.png", sizes: "128x128", type: "image/png" },
      { url: "/icons/icon-144x144.png", sizes: "144x144", type: "image/png" },
      { url: "/icons/icon-152x152.png", sizes: "152x152", type: "image/png" },
      { url: "/icons/icon-192x192.png", sizes: "192x192", type: "image/png" },
      { url: "/icons/icon-384x384.png", sizes: "384x384", type: "image/png" },
      { url: "/icons/icon-512x512.png", sizes: "512x512", type: "image/png" },
    ],
    apple: [
      { url: "/icons/icon-152x152.png", sizes: "152x152", type: "image/png" },
      { url: "/icons/icon-192x192.png", sizes: "192x192", type: "image/png" },
    ],
  },
  appleWebApp: {
    capable: true,
    statusBarStyle: "default",
    title: "يضا قم",
  },
  formatDetection: {
    telephone: false,
  },
};

export const viewport: Viewport = {
  themeColor: [
    { media: "(prefers-color-scheme: light)", color: "#10b981" },
    { media: "(prefers-color-scheme: dark)", color: "#0c1222" },
  ],
  width: "device-width",
  initialScale: 1,
  maximumScale: 1,
  userScalable: false,
};

export default function RootLayout({
  children,
}: Readonly<{
  children: React.ReactNode;
}> &#124; {
  return (
    &#124;html lang="ar" dir="rtl" suppressHydrationWarning&#124;
    &#124;head&#124;
    &#124;link rel="manifest" href="/manifest.json" /&#124;
    &#124;meta name="mobile-web-app-capable" content="yes" /&#124;
    &#124;meta name="apple-mobile-web-app-capable" content="yes" /&#124;
    &#124;meta name="apple-mobile-web-app-status-bar-style" content="default" /&#124;
    &#124;meta name="apple-mobile-web-app-title" content="يضا قم" /&#124;
    &#124;/head&#124;
    &#124;body
      className={\`${geistSans.variable} \${geistMono.variable} antialiased`}
    &#124;
    {children}
  );
}
```

```
<script
  dangerouslySetInnerHTML={{
    __html: `
      if ('serviceWorker' in navigator) {
        window.addEventListener('load', function() {
          navigator.serviceWorker.register('/sw.js').then(
            function(registration) {
              console.log('SW registered: ', registration.scope);
            },
            function(error) {
              console.log('SW registration failed: ', error);
            }
          );
        });
      }
    `,
  }}
/>
</body>
</html>
);
}
```

7. src/app/globals.css

📄 /home/z/my-project/src/app/globals.css

```
@import "tailwindcss";
@import "tw-animate-css";

@custom-variant dark (&::is(.dark *));

@theme inline {
  --color-background: var(--background);
  --color-foreground: var(--foreground);
  --font-sans: var(--font-geist-sans);
  --font-mono: var(--font-geist-mono);
  --color-sidebar-ring: var(--sidebar-ring);
  --color-sidebar-border: var(--sidebar-border);
  --color-sidebar-accent-foreground: var(--sidebar-accent-foreground);
  --color-sidebar-accent: var(--sidebar-accent);
  --color-sidebar-primary-foreground: var(--sidebar-primary-foreground);
  --color-sidebar-primary: var(--sidebar-primary);
  --color-sidebar-foreground: var(--sidebar-foreground);
  --color-sidebar: var(--sidebar);
  --color-chart-5: var(--chart-5);
  --color-chart-4: var(--chart-4);
  --color-chart-3: var(--chart-3);
  --color-chart-2: var(--chart-2);
  --color-chart-1: var(--chart-1);
  --color-ring: var(--ring);
  --color-input: var(--input);
  --color-border: var(--border);
  --color-destructive: var(--destructive);
  --color-accent-foreground: var(--accent-foreground);
  --color-accent: var(--accent);
  --color-muted-foreground: var(--muted-foreground);
  --color-muted: var(--muted);
  --color-secondary-foreground: var(--secondary-foreground);
  --color-secondary: var(--secondary);
  --color-primary-foreground: var(--primary-foreground);
  --color-primary: var(--primary);
  --color-popover-foreground: var(--popover-foreground);
  --color-popover: var(--popover);
  --color-card-foreground: var(--card-foreground);
  --color-card: var(--card);
  --radius-sm: calc(var(--radius) - 4px);
  --radius-md: calc(var(--radius) - 2px);
  --radius-lg: var(--radius);
  --radius-xl: calc(var(--radius) + 4px);

  /* Custom colors for app */
  --color-emerald-primary: var(--emerald-primary);
  --color-emerald-light: var(--emerald-light);
  --color-surface: var(--surface);
  --color-surface-hover: var(--surface-hover);
}

:root {
  --radius: 0.75rem;
  --background: #f8fafc;
  --foreground: #0f172a;
  --card: #ffffff;
  --card-foreground: #0f172a;
  --popover: #ffffff;
  --popover-foreground: #0f172a;
  --primary: #10b981;
  --primary-foreground: #ffffff;
  --secondary: #f1f5f9;
  --secondary-foreground: #475569;
  --muted: #f1f5f9;
  --muted-foreground: #64748b;
  --accent: #ecfdf5;
  --accent-foreground: #059669;
  --destructive: #ef4444;
  --border: #e2e8f0;
  --input: #e2e8f0;
  --ring: #10b981;
  --emerald-primary: #10b981;
  --emerald-light: #ecfdf5;
```

```

--surface: #ffffff;
--surface-hover: #f8fafc;
--chart-1: #10b981;
--chart-2: #3b82f6;
--chart-3: #f59e0b;
--chart-4: #ef4444;
--chart-5: #8b5cf6;
--sidebar: #ffffff;
--sidebar-foreground: #0f172a;
--sidebar-primary: #10b981;
--sidebar-primary-foreground: #ffffff;
--sidebar-accent: #ecfdf5;
--sidebar-accent-foreground: #059669;
--sidebar-border: #e2e8f0;
--sidebar-ring: #10b981;
}

.dark {
--background: #0c1222;
--foreground: #f1f5f9;
--card: #1a2235;
--card-foreground: #f1f5f9;
--popover: #1a2235;
--popover-foreground: #f1f5f9;
--primary: #10b981;
--primary-foreground: #ffffff;
--secondary: #243044;
--secondary-foreground: #cbd5e1;
--muted: #243044;
--muted-foreground: #94a3b8;
--accent: #134e4a;
--accent-foreground: #5eead4;
--destructive: #f87171;
--border: rgba(255, 255, 255, 0.1);
--input: rgba(255, 255, 255, 0.1);
--ring: #10b981;
--emerald-primary: #10b981;
--emerald-light: #134e4a;
--surface: #1a2235;
--surface-hover: #243044;
--chart-1: #34d399;
--chart-2: #60a5fa;
--chart-3: #fbbf24;
--chart-4: #f87171;
--chart-5: #a78bfa;
--sidebar: #1a2235;
--sidebar-foreground: #f1f5f9;
--sidebar-primary: #10b981;
--sidebar-primary-foreground: #ffffff;
--sidebar-accent: #134e4a;
--sidebar-accent-foreground: #5eead4;
--sidebar-border: rgba(255, 255, 255, 0.1);
--sidebar-ring: #10b981;
}

@layer base {
* {
@apply border-border outline-ring/50;
}
body {
@apply bg-background text-foreground antialiased;
font-feature-settings: "rlig" 1, "calt" 1;
}
}

/* Custom scrollbar */
::-webkit-scrollbar {
width: 8px;
height: 8px;
}

::-webkit-scrollbar-track {
background: transparent;
}

::-webkit-scrollbar-thumb {
background: #cbd5e1;
border-radius: 4px;
}

```

```

.dark::-webkit-scrollbar-thumb {
  background: #475569;
}

::-webkit-scrollbar-thumb:hover {
  background: #94a3b8;
}

.dark::-webkit-scrollbar-thumb:hover {
  background: #64748b;
}

/* Smooth transitions for theme change */
* {
  transition: background-color 0.3s ease, border-color 0.3s ease, color 0.2s ease;
}

/* Glassmorphism effect */
.glass {
  background: rgba(255, 255, 255, 0.7);
  backdrop-filter: blur(10px);
  -webkit-backdrop-filter: blur(10px);
}

.dark .glass {
  background: rgba(26, 34, 53, 0.8);
}

/* Card hover effects */
.card-hover {
  transition: all 0.3s cubic-bezier(0.4, 0, 0.2, 1);
}

.card-hover:hover {
  transform: translateY(-2px);
  box-shadow: 0 12px 24px -8px rgba(0, 0, 0, 0.15);
}

.dark .card-hover:hover {
  box-shadow: 0 12px 24px -8px rgba(0, 0, 0, 0.4);
}

/* Shimmer loading effect */
@keyframes shimmer {
  0% {
    background-position: -200% 0;
  }
  100% {
    background-position: 200% 0;
  }
}

.skeleton {
  background: linear-gradient(90deg, #e2e8f0 25%, #f1f5f9 50%, #e2e8f0 75%);
  background-size: 200% 100%;
  animation: shimmer 1.5s infinite;
}

.dark .skeleton {
  background: linear-gradient(90deg, #243044 25%, #2d3b4f 50%, #243044 75%);
  background-size: 200% 100%;
}

/* Fade in animation */
@keyframes fadeIn {
  from {
    opacity: 0;
    transform: translateY(10px);
  }
  to {
    opacity: 1;
    transform: translateY(0);
  }
}

.animate-fade-in {
  animation: fadeIn 0.4s ease-out forwards;
}

```

```

/* Scale in animation */
@keyframes scaleIn {
  from {
    opacity: 0;
    transform: scale(0.95);
  }
  to {
    opacity: 1;
    transform: scale(1);
  }
}

.animate-scale-in {
  animation: scaleIn 0.3s ease-out forwards;
}

/* Slide in from right */
@keyframes slideInRight {
  from {
    opacity: 0;
    transform: translateX(20px);
  }
  to {
    opacity: 1;
    transform: translateX(0);
  }
}

.animate-slide-in {
  animation: slideInRight 0.4s ease-out forwards;
}

/* Pulse glow effect */
@keyframes pulseGlow {
  0%, 100% {
    box-shadow: 0 0 0 0 rgba(16, 185, 129, 0.4);
  }
  50% {
    box-shadow: 0 0 0 8px rgba(16, 185, 129, 0);
  }
}

.animate-pulse-glow {
  animation: pulseGlow 2s infinite;
}

/* Gradient text */
.gradient-text {
  background: linear-gradient(135deg, #10b981 0%, #059669 50%, #047857 100%);
  -webkit-background-clip: text;
  -webkit-text-fill-color: transparent;
  background-clip: text;
}

/* Button press effect */
.btn-press {
  transition: all 0.15s ease;
}

.btn-press:active {
  transform: scale(0.97);
}

/* Floating animation */
@keyframes float {
  0%, 100% {
    transform: translateY(0);
  }
  50% {
    transform: translateY(-5px);
  }
}

.animate-float {
  animation: float 3s ease-in-out infinite;
}

/* Progress bar animation */
@keyframes progress {

```

```

    from {
        width: 0%;
    }
}

.animate-progress {
    animation: progress 1s ease-out forwards;
}

/* Checkbox animation */
@keyframes checkmark {
    0% {
        stroke-dashoffset: 100;
    }
    100% {
        stroke-dashoffset: 0;
    }
}

.checkmark-animate {
    stroke-dasharray: 100;
    stroke-dashoffset: 100;
    animation: checkmark 0.4s ease-out forwards;
}

/* Modal backdrop blur */
.modal-backdrop {
    backdrop-filter: blur(4px);
    -webkit-backdrop-filter: blur(4px);
}

/* Input focus ring */
.input-focus:focus {
    outline: none;
    box-shadow: 0 0 0 3px rgba(16, 185, 129, 0.2);
}

/* Badge shine effect */
@keyframes shine {
    0% {
        left: -100%;
    }
    100% {
        left: 100%;
    }
}

.badge-shine {
    position: relative;
    overflow: hidden;
}

.badge-shine::after {
    content: '';
    position: absolute;
    top: 0;
    left: -100%;
    width: 100%;
    height: 100%;
    background: linear-gradient(90deg, transparent, rgba(255, 255, 255, 0.3), transparent);
    animation: shine 2s infinite;
}

```


8. src/app/api/auth/register/route.ts

□ /home/z/my-project/src/app/api/auth/register/route.ts

```
import { NextRequest, NextResponse } from 'next/server'
import { prisma } from '@lib/prisma'
import { randomBytes } from 'crypto'

// رورمل اءملكل طيسب ريشت
const hashPassword = (password: string): string => {
  let hash = 0
  for (let i = 0; i < password.length; i++) {
    const char = password.charCodeAt(i)
    hash = ((hash << 5) - hash) + char
    hash = hash & hash
  }
  return hash.toString(36) + randomBytes(16).toString('hex')
}

// سئلجلا زمر ديوت
const generateSessionToken = () => randomBytes(32).toString('hex')

export async function POST(request: NextRequest) {
  try {
    const { name, email, password } = await request.json()

    if (!name || !email || !password) {
      return NextResponse.json({ error: 'ءبولطم لوقحلا عيمح' }, { status: 400 })
    }

    if (password.length < 6) {
      return NextResponse.json({ error: 'لقالا دلع فرحأ 6 نوكت نأ بجي رورمل اءملك' }, { status: 400 })
    }

    // باسئلجلا دوجو مدع نم ققحتلا
    const existingUser = await prisma.user.findUnique({
      where: { email: email.toLowerCase() }
    })

    if (existingUser) {
      return NextResponse.json({ error: 'اقبسمل لءسم ينورتكلللا ديربالا' }, { status: 400 })
    }

    // باسئلجلا اشن
    const hashedPassword = hashPassword(password)
    const user = await prisma.user.create({
      data: {
        name,
        email: email.toLowerCase(),
        password: hashedPassword,
      }
    })

    // سئلجلا اشن
    const sessionToken = generateSessionToken()
    const expiresAt = new Date()
    expiresAt.setDate(expiresAt.getDate() + 30) // 30 موي

    await prisma.session.create({
      data: {
        id: sessionToken,
        userId: user.id,
        expiresAt
      }
    })

    const response = NextResponse.json({
      user: { id: user.id, name: user.name, email: user.email, avatar: user.avatar }
    })

    // سئلجلا يكوك نيغت
    response.cookies.set('session-token', sessionToken, {
      httpOnly: true,
      secure: process.env.NODE_ENV === 'production',
      sameSite: 'lax',
      maxAge: 60 * 60 * 24 * 30 // 30 موي
    })
  }
}
```

```
    return response
  } catch (error) {
    console.error('Register error:', error)
    return NextResponse.json({ error: 'بأسجلء اشن إف أطخ ثدح' }, { status: 500 })
  }
}
```

9. src/app/api/auth/login/route.ts

📄 /home/z/my-project/src/app/api/auth/login/route.ts

```
import { NextRequest, NextResponse } from 'next/server'
import { prisma } from '@lib/prisma'
import { randomBytes } from 'crypto'

// رورمل ةملك نم ققحتلا
const verifyPassword = (password: string, hashedPassword: string): boolean => {
  let hash = 0
  for (let i = 0; i < password.length; i++) {
    const char = password.charCodeAt(i)
    hash = ((hash << 5) - hash) + char
    hash = hash & hash
  }
  return hashedPassword.startsWith(hash.toString(36))
}

// ةسلجل زمر ديلوت
const generateSessionToken = () => randomBytes(32).toString('hex')

export async function POST(request: NextRequest) {
  try {
    const { email, password } = await request.json()

    if (!email || !password) {
      return NextResponse.json({ error: 'نابولطم رورمل ةملك وينورتكلل ديربل' }, { status: 400 })
    }

    // مدختسمل نع ثحبلا
    const user = await prisma.user.findUnique({
      where: { email: email.toLowerCase() }
    })

    if (!user) {
      return NextResponse.json({ error: 'لجسم ريغ وينورتكلل ديربل' }, { status: 401 })
    }

    // رورمل ةملك نم ققحتلا
    if (!verifyPassword(password, user.password)) {
      return NextResponse.json({ error: 'ةحيص ريغ رورمل ةملك' }, { status: 401 })
    }

    // ةديج ةسلج ءاشن
    const sessionToken = generateSessionToken()
    const expiresAt = new Date()
    expiresAt.setDate(expiresAt.getDate() + 30)

    // ةددعتم ةزهج نم لوخدلا ليجسبت حامسلل (ىرخال تاسلجل فذح نودب ةديج ةسلج ءاشن)
    await prisma.session.create({
      data: {
        id: sessionToken,
        userId: user.id,
        expiresAt
      }
    })

    const response = NextResponse.json({
      user: { id: user.id, name: user.name, email: user.email, avatar: user.avatar }
    })

    response.cookies.set('session-token', sessionToken, {
      httpOnly: true,
      secure: process.env.NODE_ENV === 'production',
      sameSite: 'lax',
      maxAge: 60 * 60 * 24 * 30
    })

    return response
  } catch (error) {
    console.error('Login error:', error)
    return NextResponse.json({ error: 'لوخدلا ليجسبت يف أطح ثدح' }, { status: 500 })
  }
}
```

10. src/app/api/auth/logout/route.ts

□ /home/z/my-project/src/app/api/auth/logout/route.ts

```
import { NextRequest, NextResponse } from 'next/server'
import { prisma } from '@lib/prisma'

export async function POST(request: NextRequest) {
  try {
    const sessionToken = request.cookies.get('session-token')?.value

    if (sessionToken) {
      // تانايبل اءءءاق نم ءءلءل فءء
      await prisma.session.deleteMany({
        where: { id: sessionToken }
      })
    }

    const response = NextResponse.json({ success: true })
    response.cookies.delete('session-token')

    return response
  } catch (error) {
    console.error('Logout error:', error)
    return NextResponse.json({ error: 'أطء ءءء' }, { status: 500 })
  }
}
```

11. src/app/api/auth/me/route.ts

📄 /home/z/my-project/src/app/api/auth/me/route.ts

```
import { NextRequest, NextResponse } from 'next/server'
import { prisma } from '@lib/prisma'

export async function GET(request: NextRequest) {
  try {
    const sessionToken = request.cookies.get('session-token')?.value

    if (!sessionToken) {
      return NextResponse.json({ error: 'لودل لجسم ريغ' }, { status: 401 })
    }

    // سولجل نغ ثحبلا
    const session = await prisma.session.findFirst({
      where: {
        id: sessionToken,
        expiresAt: { gt: new Date() }
      },
      include: { user: true }
    })

    if (!session) {
      // حيالصل اةيهتنملا تاسلجللا فيظنت
      await prisma.session.deleteMany({
        where: {
          expiresAt: { lt: new Date() }
        }
      }).catch(() => {})

      return NextResponse.json({ error: 'ةيهتنم ةسلجللا' }, { status: 401 })
    }

    return NextResponse.json({
      user: {
        id: session.user.id,
        name: session.user.name,
        email: session.user.email,
        avatar: session.user.avatar
      }
    })
  } catch (error) {
    console.error('Get user error:', error)
    return NextResponse.json({ error: 'أطخ ثدح' }, { status: 500 })
  }
}
```

12. src/app/api/sync/route.ts

```
□ /home/z/my-project/src/app/api/sync/route.ts
```

```
import { NextRequest, NextResponse } from 'next/server'
import { prisma } from '@lib/prisma'

// جلب المستخدم الحالي من الجلسة
async function getCurrentUser(request: NextRequest) {
  const sessionToken = request.cookies.get('session-token')?.value

  if (!sessionToken) return null

  const session = await prisma.session.findFirst({
    where: {
      id: sessionToken,
      expiresAt: { gt: new Date() }
    },
    include: { user: true }
  })

  return session?.user || null
}

// GET - تانايي لي محت
export async function GET(request: NextRequest) {
  try {
    const user = await getCurrentUser(request)
    if (!user) {
      return NextResponse.json({ error: 'لؤخدلا لجسم ريغ' }, { status: 401 })
    }

    // مخرجتسملا تانايي عي مج بلج
    const [items, familyMembers, customStores, priceHistory, budget, customCategories] = await Promise.all([
      prisma.item.findMany({
        where: { userId: user.id },
        include: { prices: true }
      }),
      prisma.familyMember.findMany({
        where: { userId: user.id }
      }),
      prisma.customStore.findMany({
        where: { userId: user.id }
      }),
      prisma.priceHistoryRecord.findMany({
        where: { userId: user.id }
      }),
      prisma.budget.findUnique({
        where: { userId: user.id }
      }),
      prisma.customCategory.findMany({
        where: { userId: user.id }
      })
    ])

    // (ادوجوم لودجل نوكي ال دق) ةطوفجمل تاحارتقالا بلج ةلواجم
    let savedProductNames: string[] = []
    try {
      const saved = await prisma.savedProductName.findMany({
        where: { userId: user.id }
      })
      savedProductNames = saved.map(s => s.name)
    } catch {
      // دعب دوجوم ريغ لودجل
    }

    // جتمل مسا بسج راعسأل قيسنت
    const formattedPriceHistory: Record<string, { store: string; price: number; date: string }[]> = {}
    for (const record of priceHistory) {
      const key = record.productName.toLowerCase()
      if (!formattedPriceHistory[key]) formattedPriceHistory[key] = []
      formattedPriceHistory[key].push({
        store: record.store,
        price: record.price,
        date: record.date.toISOString()
      })
    }
  }
}
```

```

// صاوغ ألأ قوسنآ
const formattedItems = items.map(item => ({
  id: item.id,
  name: item.name,
  category: item.category,
  quantity: item.quantity,
  notes: item.notes,
  isPurchased: item.isPurchased,
  image: item.image,
  prices: item.prices.map(p => ({
    store: p.store,
    price: p.price,
    date: p.date.toISOString()
  })),
  createdAt: item.createdAt.toISOString()
}))

// لؤلألأ دارفأ قوسنآ
const formattedFamilyMembers = familyMembers.map(m => ({
  id: m.id,
  name: m.name,
  avatar: m.avatar,
  email: ''
}))

// صمءم لآ نأفوسنآ قوسنآ
const formattedCustomCategories = customCategories.map(c => ({
  id: c.id,
  name: c.name,
  icon: c.icon,
  color: c.color,
  keywords: c.keywords ? c.keywords.split(',').map(k => k.trim()).filter(k => k) : []
}))

return NextResponse.json({
  items: formattedItems,
  familyMembers: formattedFamilyMembers,
  customStores: customStores.map(s => s.name),
  priceHistory: formattedPriceHistory,
  budget: budget ? {
    monthlyBudget: budget.monthlyBudget,
    spentAmount: budget.spentAmount,
    startDate: budget.startDate?.toISOString() || '',
    shoppingTurn: budget.shoppingTurn || ''
  } : null,
  customCategories: formattedCustomCategories,
  savedProductNames
})
} catch (error) {
  console.error('Sync get error:', error)
  return NextResponse.json({ error: 'أطء ءءء' }, { status: 500 })
}
}

// POST - مءءسم لآ نأفوسنآ طفف
export async function POST(request: NextRequest) {
  try {
    const user = await getCurrentUser(request)
    if (!user) {
      return NextResponse.json({ error: 'لوءءلآ لءسم روء' }, { status: 401 })
    }

    const { items, familyMembers, customStores, priceHistory, budget, customCategories, savedProductNames } = await request.json()

    // (لءءءء - savedProductNames) ءمءءلآ نأفوسنآ طفف
    await Promise.all([
      prisma.item.deleteMany({ where: { userId: user.id } }),
      prisma.familyMember.deleteMany({ where: { userId: user.id } }),
      prisma.customStore.deleteMany({ where: { userId: user.id } }),
      prisma.priceHistoryRecord.deleteMany({ where: { userId: user.id } }),
      prisma.customCategory.deleteMany({ where: { userId: user.id } }),
    ])

    // صاوغ ألأ طفف
    if (items && items.length > 0) {
      for (const item of items) {
        await prisma.item.create({
          data: {
            id: item.id,

```

```

        name: item.name,
        category: item.category,
        quantity: item.quantity || 1,
        notes: item.notes || '',
        isPurchased: item.isPurchased || false,
        image: item.image || null,
        userId: user.id,
        prices: {
          create: (item.prices || []).map((p: { store: string; price: number; date: string }) => ({
            store: p.store,
            price: p.price,
            date: new Date(p.date)
          })))
        }
      }
    })
  }
}

// إنشاء العائلات
if (familyMembers && familyMembers.length > 0) {
  await prisma.familyMember.createMany({
    data: familyMembers.map((m: { id: string; name: string; avatar: string }) => ({
      id: m.id,
      name: m.name,
      avatar: m.avatar || '',
      userId: user.id
    })))
}

// إنشاء المحلات
if (customStores && customStores.length > 0) {
  await prisma.customStore.createMany({
    data: customStores.map((name: string) => ({
      name,
      userId: user.id
    })))
}

// تسجيل الأسعار
if (priceHistory && Object.keys(priceHistory).length > 0) {
  const records: { productName: string; store: string; price: number; date: Date; userId: string }[] = []
  for (const [productName, prices] of Object.entries(priceHistory)) {
    for (const p of (prices as { store: string; price: number; date: string }[])) {
      records.push({
        productName,
        store: p.store,
        price: p.price,
        date: new Date(p.date),
        userId: user.id
      })
    }
  }
  if (records.length > 0) {
    await prisma.priceHistoryRecord.createMany({ data: records })
  }
}

// إنشاء الميزانية
if (budget) {
  await prisma.budget.upsert({
    where: { userId: user.id },
    update: {
      monthlyBudget: budget.monthlyBudget || 0,
      spentAmount: budget.spentAmount || 0,
      startDate: budget.startDate ? new Date(budget.startDate) : null,
      shoppingTurn: budget.shoppingTurn || null
    },
    create: {
      monthlyBudget: budget.monthlyBudget || 0,
      spentAmount: budget.spentAmount || 0,
      startDate: budget.startDate ? new Date(budget.startDate) : null,
      shoppingTurn: budget.shoppingTurn || null,
      userId: user.id
    }
  })
}
}

```



```

// مخصص المصنفات التي لم تضاف
if (customCategories && customCategories.length > 0) {
  await prisma.customCategory.createMany({
    data: customCategories.map((cat: { id: string; name: string; icon: string; color: string; keywords: string[] }) => ({
      id: cat.id,
      name: cat.name,
      icon: cat.icon || '',
      color: cat.color || 'gray',
      keywords: Array.isArray(cat.keywords) ? cat.keywords.join(',') : '',
      userId: user.id
    })))
  })
}

// دمج الفئات - (تحويل الفئات إلى فئات واحدة)
if (savedProductNames && Array.isArray(savedProductNames)) {
  for (const name of savedProductNames) {
    if (name && typeof name === 'string' && name.trim()) {
      try {
        await prisma.savedProductName.create({
          data: {
            name: name.trim(),
            userId: user.id
          }
        })
      } catch {
        // راركتلا أخطأ لهاجت (unique constraint)
      }
    }
  }
}

return NextResponse.json({ success: true })
} catch (error) {
  console.error('Sync post error:', error)
  const errorMessage = error instanceof Error ? error.message : 'أخطأ حدث'
  return NextResponse.json({ error: errorMessage }, { status: 500 })
}
}

```

13. src/app/api/suggestions/delete/route.ts

```
📄 /home/z/my-project/src/app/api/suggestions/delete/route.ts
```

```
import { NextRequest, NextResponse } from 'next/server'
import { prisma } from '@lib/prisma'

// جلب المستخدم الحالي من دatasession
async function getCurrentUser(request: NextRequest) {
  const sessionToken = request.cookies.get('session-token')?.value

  if (!sessionToken) return null

  const session = await prisma.session.findFirst({
    where: {
      id: sessionToken,
      expiresAt: { gt: new Date() }
    },
    include: { user: true }
  })

  return session?.user || null
}

// DELETE - حذف منتج من قائمة المقترحات
export async function DELETE(request: NextRequest) {
  try {
    const user = await getCurrentUser(request)
    if (!user) {
      return NextResponse.json({ error: 'لقد دلت على حساب غير صالح' }, { status: 401 })
    }

    const body = await request.json()
    const productName = body.productName

    if (!productName || typeof productName !== 'string') {
      return NextResponse.json({ error: 'الاسم غير صحيح' }, { status: 400 })
    }

    const productNameLower = productName.toLowerCase().trim()

    console.log(`📝 حذف منتج: "${productName}" (${productNameLower}) من قبل المستخدم: ${user.id}`)

    let savedNamesDeleted = 0
    let priceHistoryDeleted = 0
    let itemsDeleted = 0

    // 1. حذف المقترحات (SavedProductName)
    try {
      const allSavedNames = await prisma.savedProductName.findMany({
        where: { userId: user.id },
        select: { id: true, name: true }
      })
      const savedNameIdsToDelete = allSavedNames
        .filter(s => s.name.toLowerCase().trim() === productNameLower)
        .map(s => s.id)

      if (savedNameIdsToDelete.length > 0) {
        await prisma.savedProductName.deleteMany({
          where: { id: { in: savedNameIdsToDelete } }
        })
        savedNamesDeleted = savedNameIdsToDelete.length
      }
    } catch (e) {
      console.log('SavedProductName table might not exist yet:', e)
    }

    // 2. حذف سجلات التاريخ (PriceHistoryRecord)
    try {
      const deletedPriceHistory = await prisma.priceHistoryRecord.deleteMany({
        where: {
          userId: user.id,
          productName: productNameLower
        }
      })
      priceHistoryDeleted = deletedPriceHistory.count
    } catch (e) {
    }
  }
}
```

```

    console.log('PriceHistoryRecord delete error:', e)
  }

  // 3. مسالال سفن لمحت يتللا (Items) ضارغلألا نم فذح
  try {
    const allItems = await prisma.item.findMany({
      where: { userId: user.id },
      select: { id: true, name: true }
    })

    const itemIdsToDelete = allItems
      .filter(item => item.name.toLowerCase().trim() === productNameLower)
      .map(item => item.id)

    if (itemIdsToDelete.length > 0) {
      const deletedItems = await prisma.item.deleteMany({
        where: {
          id: { in: itemIdsToDelete }
        }
      })
      itemsDeleted = deletedItems.count
    }
  } catch (e) {
    console.log('Item delete error:', e)
  }

  console.log(`\n:تاح ارتقالا نم "${productName}" فذح مت`, {
    savedNames: savedNamesDeleted,
    priceHistory: priceHistoryDeleted,
    items: itemsDeleted
  })

  return NextResponse.json({
    success: true,
    deleted: {
      savedNames: savedNamesDeleted,
      priceHistory: priceHistoryDeleted,
      items: itemsDeleted
    }
  })
} catch (error) {
  console.error('Delete suggestion error:', error)
  return NextResponse.json({ error: 'فذللا يف أطخ نذح' }, { status: 500 })
}
}

```

14. src/app/api/classify/route.ts

```
❏ /home/z/my-project/src/app/api/classify/route.ts
```

```
import ZAI from 'z-ai-web-dev-sdk';
import { NextResponse, NextRequest } from 'next/server';
import { prisma } from '@lib/prisma';

// تصنيفات الطعام في رستوراننا
const CATEGORIES = [
  {
    id: 'dairy',
    name: 'منتجات الألبان',
    keywords: [
      'دبزل', 'دبزل', 'ميرك', 'مطشيق', 'عطرشق', 'بور', 'ديابز', 'نبح', 'نبل', 'بيج', 'كبوكرام', 'كوب', 'يريك', 'اليزازوم', 'رديش', 'نابجا', 'روم', 'كشك', 'نبل', 'نشينراك', 'هلتسن', 'ودين', 'وردوب بيلج', 'فجحم بيلج', 'زوتكال'
    ]
  },
  {
    id: 'meat',
    name: 'لحوم',
    keywords: [
      'فورخ', 'فورخ', 'ناض', 'ناض', 'لجع', 'يرقب', 'رقب', 'همجل', 'همجل', 'محجل', 'تازوم', 'شير', 'كيتس', 'هورقم همجل', 'هورقم محجل', 'هتفك', 'هتفك', 'باباك', 'اليدترم', 'نوشنال', 'للخم', 'يلجع محجل', 'منغ محجل', 'مطعب محجل', 'قربه'
    ]
  },
  {
    id: 'poultry',
    name: 'طيور',
    keywords: [
      'جاجد رودص', 'عطقم جاجد', 'جورف', 'خراف', 'خارف', 'هجاجد', 'هجاجد', 'جاجا', 'يخ جاجد', 'باقر', 'صناوق', 'هدبك', 'ديك', 'هجنجأ', 'هجنجأ', 'ذافأ', 'زوأ', 'هزو', 'هزو', 'زو', 'هطب', 'هطب', 'طب', 'يمور', 'يموريك', 'ليد'
    ]
  },
  {
    id: 'seafood',
    name: 'سمك',
    keywords: [
      'ميرش', 'يربمج', 'نايبور', 'نايبور', 'كامسا', 'كمس', 'كمس', 'شيف', 'روماه', 'دغنك', 'صاب', 'روبص', 'روعش', 'لجان', 'روماه', 'بقبق', 'نوملس', 'نوملاس', 'ليركام', 'هنوت', 'هنوت', 'نديرس', 'راجم', 'طوبطحأ', 'طيس', 'يراملالك', 'رابج', 'ازوكاسا', 'دنكر'
    ]
  },
  {
    id: 'bakery',
    name: 'خبز',
    keywords: [
      'نوساورك', 'يلوماص', 'نومص', 'رمسأ زبخ', 'رب زبخ', 'صيأ زبخ', 'زبخ', 'يكرت زبخ', 'يناري زبخ', 'يبرع زبخ', 'تسوت زبخ', 'تسوت', 'اوساورك', 'يسينرف زبخ', 'توكسيب', 'تامسيق', 'طامسيق', 'اليتروت', 'جاص زبخ', 'رياطف', 'ريطف', 'صمحم زبخ', 'لومعم', 'كاك', 'كعك', 'وني', 'نوتاب'
    ]
  },
  {
    id: 'vegetables',
    name: 'خضروات',
    keywords: [
      'اطاطب', 'سطاطب', 'موث', 'لمب', 'هرودنب', 'مطامط', 'مطامط', 'تاوريض', 'راصح', 'ناولأ لفل', 'يمور لفل', 'راح لفل', 'لفل', 'رايح', 'رايح', 'هرنج', 'رنج', 'يلسس عرق', 'عرق', 'هسوك', 'هسوك', 'اسوك', 'هانجنزاب', 'نانجنزاب', 'ارصح ايلوصاف', 'هيماب', 'هيما', 'هيخولم', 'هيخولم', 'خنابس', 'ريجج', 'سخ', 'هربزك', 'سينودقب', 'تبش', 'سفرك', 'تارك', 'رضجأ لوف', 'علسب', 'اءالزاب', 'تفل', 'لجف', 'بنرك', 'فوقلم', 'طيينرق', 'اهرز', 'ناجير', 'عانعن', 'سناك', 'مورشم', 'رطف', 'هرذ', 'ينادوس لوف', 'هلج اطاطب', 'ردنمش'
    ]
  },
  {
    id: 'fruits',
    name: 'فاكهة',
    keywords: [
      'روزم', 'هوزم', 'زوم', 'هافت', 'هافت', 'هافت', 'هكاف', 'هكاف', 'هكاف', 'رفصأ نوميل', 'رضجأ نوميل', 'نوميل', 'نيردنم', 'يفسوي', 'ناقترپ', 'لاقترپ', 'سانانأ', 'اجنم', 'وجنام', 'يويك', 'وداكوا', 'وداكوا', 'تورف بيرج'
    ]
  }
];
```

[illegible]

```

    'قوسم', 'لوسغ', 'لئاس نوباص', 'نوباص', 'فطنم', 'تافطنم',
    'يريف', 'قيرب', 'يريف', 'واد', 'ديات', 'لييرا', 'ليسرب', 'ليسغ قوسم',
    'عملم', 'تامزيم', 'ميبم', 'لوتد', 'لوتيد', 'رهطم', 'ليفاج', 'رولك',
    'ليدانم', 'طوف', 'طوف', 'جنقسا', 'هسقم', 'هسقم', 'هسقم', 'هسقم',
    'ديس', 'ليسرب', 'ديات', 'سلينبوب', 'ومو', 'اليس', 'شيداف', 'لندنم',
    'تفوس', 'رونيل', 'تروفموك', 'ينواد', 'سبالم رطعم', 'وج رطعم', 'رطعم',
    'فود', 'وكود', 'هاسرف', 'هاسرف', 'فيطنط طوف', 'زافق', 'تازافق'
  ],
},
{
  id: 'personal',
  name: 'هيمخش هيانع',
  keywords: [
    'conditioner', 'مسلب', 'رعش وبماش', 'وبماش', 'هيمخش هيانع', 'هيمخش هيانع',
    'يبط نوباص', 'مسج لوسغ', 'هجو لوسغ', 'نشول', 'بطرم ميرك', 'ميرك',
    'نيرتسيل', 'مف لوسغ', 'نانسا هاشرف', 'نانسا هاشرف', 'نوجعم', 'نانسا نوجعم',
    'عقالح ليح', 'تارفش', 'عقالح ميرك', 'لور', 'خاخب', 'تارودود', 'قرع ليزم',
    'تاعضرم', 'زربماب', 'ضافح', 'تاضافح', 'هيجص طوف', 'هيجص ليدانم', 'مراجم',
    'يويح داصم', 'نكسم', 'نيماتيغ', 'ءاود', 'تافاعسا', 'طابر', 'شاش', 'نطق',
    'وردوب', 'ساسا ميرك', 'جور', 'اراكسام', 'جايكم', 'يسميش قياو', 'يقاو ميرك'
  ],
},
{
  id: 'kitchen',
  name: 'تاودأو خبطم',
  keywords: [
    'يناوأ', 'رودق مقط', 'مقط', 'هعالقلم', 'هعالقلم', 'رودق', 'ردق', 'خبطم',
    'تاساك', 'سوؤك', 'بوك', 'قابطا', 'قبط', 'نوحص', 'نحس', 'ءان',
    'نيكالكس', 'ءدام', 'نيكالكس', 'نيكس', 'كوش', 'هكوش', 'قعالم', 'هقعلم',
    'ماعط رضحم', 'هناوحاط', 'ءرامع', 'طالح', 'فيووركيم', 'نرف', 'ءجالث',
    'هجورم', 'هسينكم', 'هئيابرهك سينكم', 'ءفاشن', 'هلاسغ', 'مماك', 'هئالغ',
    'تادخم', 'دئاسو', 'هيطغا', 'تايناطق', 'تايالم', 'فشارش', 'رئاتس', 'داجس'
  ],
},
{
  id: 'other',
  name: 'يرخأ',
  keywords: []
}
];

export async function POST(request: NextRequest) {
  try {
    const zai = await ZAI.create();
    const { itemName, customCategories } = await request.json();

    if (!itemName || typeof itemName !== 'string') {
      return NextResponse.json({ error: 'بولطم ضرغل مسأ' }, { status: 400 });
    }

    // قيقدو عيرس "الوأ هيجاتفملا تاملكلا يف ثحبلا
    const itemNameLower = itemName.toLowerCase();
    let keywordCategory: string | null = null;

    // هيجارتقالا تافينمتملا يف ثحبلا
    for (const cat of CATEGORIES) {
      if (cat.keywords.some(keyword => itemNameLower.includes(keyword))) {
        keywordCategory = cat.id;
        break;
      }
    }

    // مصيخملا يف ثحبن، هيجارتقالا يف دجن مل اذإ
    if (!keywordCategory && customCategories && Array.isArray(customCategories)) {
      for (const cat of customCategories) {
        if (cat.keywords && Array.isArray(cat.keywords) && cat.keywords.length > 0) {
          if (cat.keywords.some((keyword: string) => itemNameLower.includes(keyword.toLowerCase()))) {
            keywordCategory = cat.id;
            // ةرشابم مصيخملا فينمتملا عجرن
            return NextResponse.json({
              category: cat.id,
              categoryName: cat.name,
              method: 'custom_keywords'
            });
          }
        }
      }
    }
  }
}

```

```

// مدمختسن ، ةيحاتفملا تاملكلا نم فينصت اندجو اذ
if (keywordCategory) {
  const categoryInfo = CATEGORIES.find(c => c.id === keywordCategory);
  return NextResponse.json({
    category: keywordCategory,
    categoryName: categoryInfo?.name || 'ىخ',
    method: 'keywords'
  });
}

// للى ةيضايرتفالا عم ةمصملا تافينصتلا جم د
const allCategories = [...CATEGORIES];
if (customCategories && Array.isArray(customCategories)) {
  for (const cat of customCategories) {
    allCategories.push({
      id: cat.id,
      name: cat.name,
      keywords: cat.keywords || []
    });
  }
}

// قباطت دجن مل اذ فينصتلا AI مادختسا
const categoriesDescription = allCategories.map(c =>
  ` - ${c.id}: ${c.name}${c.keywords && c.keywords.length > 0 ? ' (لثم: ' + c.keywords.slice(0, 3).join(', ') + ')`
).join('\n');

const completion = await zai.chat.completions.create({
  messages: [
    {
      role: 'system',
      content: `ةحاتملا تافينصتلا . ةببرعلا تاييرتشملا تاجتنم فينصتل يكذ دغاسم تنأ:
${categoriesDescription}`
    },
    {
      role: 'user',
      content: `صرغلا اذه فنص: "${itemName}"`
    }
  ],
  temperature: 0.1,
  max_tokens: 20
});

const aiCategory = completion.choices[0]?.message?.content?.trim().toLowerCase() || 'other';

// فينصتلا ةحص نم ققحتلا
const validCategories = allCategories.map(c => c.id);
const finalCategory = validCategories.includes(aiCategory) ? aiCategory : 'other';

// مصم و ا يضايرتفا فينصتلا تامولعم نع ثحبلا
let categoryInfo = CATEGORIES.find(c => c.id === finalCategory);
let categoryName = categoryInfo?.name || 'ىخ';

if (!categoryInfo && customCategories && Array.isArray(customCategories)) {
  const customCat = customCategories.find((c: { id: string; name: string }) => c.id === finalCategory);
  if (customCat) {
    categoryName = customCat.name;
  }
}

return NextResponse.json({
  category: finalCategory,
  categoryName: categoryName,
  method: 'ai'
});

} catch (error) {
  console.error('Classification error:', error);
  return NextResponse.json(
    { error: 'فينصتلا يف اطح ثدج', category: 'other', categoryName: 'ىخ' },
    { status: 500 }
  );
}
}

```

15. src/app/api/family/route.ts

```
□ /home/z/my-project/src/app/api/family/route.ts
```

```
import { NextRequest, NextResponse } from 'next/server'
import { prisma } from '@lib/prisma'
import { randomBytes } from 'crypto'

// جلب المستخدم الحالي
async function getCurrentUser(request: NextRequest) {
  const sessionToken = request.cookies.get('session-token')?.value
  if (!sessionToken) return null

  const session = await prisma.session.findFirst({
    where: {
      id: sessionToken,
      expiresAt: { gt: new Date() }
    },
    include: { user: true }
  })

  return session?.user || null
}

// توليد كود دعوة
const generateInviteCode = () => {
  return randomBytes(4).toString('hex').toUpperCase()
}

// GET - جلب قائمة المستخدمين
export async function GET(request: NextRequest) {
  try {
    const user = await getCurrentUser(request)
    if (!user) {
      return NextResponse.json({ error: 'لست مجاز' }, { status: 401 })
    }

    // جلب المستخدم مع العائلة
    const userWithFamily = await prisma.user.findUnique({
      where: { id: user.id },
      include: {
        family: {
          include: {
            members: {
              select: {
                id: true,
                name: true,
                email: true,
                avatar: true,
                familyRole: true
              }
            }
          }
        }
      }
    })

    // جلب قائمة الدعايات
    let invitations = []
    if (userWithFamily?.family) {
      invitations = await prisma.familyInvitation.findMany({
        where: {
          familyId: userWithFamily.family.id,
          status: 'pending',
          expiresAt: { gt: new Date() }
        }
      })
    }

    return NextResponse.json({
      user: {
        id: user.id,
        name: user.name,
        email: user.email,
        avatar: user.avatar,
        familyId: userWithFamily?.familyId,
        familyRole: userWithFamily?.familyRole
      },
      invitations
    })
  }
}
```



```

    },
    family: userWithFamily?.family ? {
      id: userWithFamily.family.id,
      name: userWithFamily.family.name,
      inviteCode: userWithFamily.family.inviteCode,
      members: userWithFamily.family.members
    } : null,
    invitations
  })
} catch (error) {
  console.error('Get family error:', error)
  return NextResponse.json({ error: 'أخطأ شئ' }, { status: 500 })
}
}

// POST - مامضنالا وأ ةديج ةلئاع ءاشن
export async function POST(request: NextRequest) {
  try {
    const user = await getCurrentUser(request)
    if (!user) {
      return NextResponse.json({ error: 'لوخدلا لجسم ريغ' }, { status: 401 })
    }

    const body = await request.json()
    const { action, familyName, inviteCode } = body

    if (action === 'create') {
      // ةديج ةلئاع ءاشن
      const inviteCode = generateInviteCode()

      const family = await prisma.family.create({
        data: {
          name: familyName || `ةلئاع ${user.name}`,
          inviteCode,
          members: {
            connect: { id: user.id }
          }
        }
      })

      // مدخسمل رود شيدجت
      await prisma.user.update({
        where: { id: user.id },
        data: {
          familyId: family.id,
          familyRole: 'owner'
        }
      })

      return NextResponse.json({
        success: true,
        family: {
          id: family.id,
          name: family.name,
          inviteCode: family.inviteCode
        }
      })
    }

    if (action === 'join') {
      // ةوعدلا دوك ربع ةلئاعل مامضنالا
      const family = await prisma.family.findUnique({
        where: { inviteCode }
      })

      if (!family) {
        return NextResponse.json({ error: 'حجص ريغ ةوعدلا دوك' }, { status: 400 })
      }

      // اقبسمل مدخسمل مامضنالا مدع نم ققحتلا
      const existingMember = await prisma.user.findFirst({
        where: {
          id: user.id,
          familyId: family.id
        }
      })

      if (existingMember) {
        return NextResponse.json({ error: 'ةلئاعلا هذو يف لعفلاب وضع تنأ' }, { status: 400 })
      }
    }
  }
}

```

```

    }

    // جلب العائلة
    await prisma.user.update({
      where: { id: user.id },
      data: {
        familyId: family.id,
        familyRole: 'member'
      }
    })

    return NextResponse.json({
      success: true,
      family: {
        id: family.id,
        name: family.name
      }
    })
  }

  if (action === 'invite') {
    // تحديث موعد الشن
    const currentUser = await prisma.user.findUnique({
      where: { id: user.id },
      include: { family: true }
    })

    if (!currentUser?.family) {
      return NextResponse.json({ error: 'الوأة لعائاع فف ءاوضع نوكف نأ بءف' }, { status: 400 })
    }

    if (currentUser.familyRole === 'member') {
      return NextResponse.json({ error: 'ءوعءلا ءفءالص لفءل سفل' }, { status: 403 })
    }

    const code = generateInviteCode()
    const expiresAt = new Date()
    expiresAt.setDate(expiresAt.getDate() + 7) // عوبسأ ءعب فءفنف

    const invitation = await prisma.familyInvitation.create({
      data: {
        code,
        email: body.email || '',
        familyId: currentUser.family.id,
        invitedBy: user.id,
        role: body.role || 'member',
        expiresAt
      }
    })

    return NextResponse.json({
      success: true,
      invitation: {
        id: invitation.id,
        code: invitation.code,
        expiresAt: invitation.expiresAt
      }
    })
  }

  if (action === 'leave') {
    // جلب العءاءم
    await prisma.user.update({
      where: { id: user.id },
      data: {
        familyId: null,
        familyRole: 'member'
      }
    })

    return NextResponse.json({ success: true })
  }

  return NextResponse.json({ error: 'فوءعم رفء ءارء' }, { status: 400 })
} catch (error) {
  console.error('Family action error:', error)
  return NextResponse.json({ error: 'أطء ءءء' }, { status: 500 })
}
}

```

16. src/app/api/invite/route.ts

□ /home/z/my-project/src/app/api/invite/route.ts

```
import { NextRequest, NextResponse } from 'next/server'
import { prisma } from '@lib/prisma'
import { randomBytes } from 'crypto'

// لاجل المدخلات المدخلة الى لوجين
async function getCurrentUser(request: NextRequest) {
  const sessionToken = request.cookies.get('session-token')?.value
  if (!sessionToken) return null

  const session = await prisma.session.findFirst({
    where: {
      id: sessionToken,
      expiresAt: { gt: new Date() }
    },
    include: { user: true }
  })

  return session?.user || null
}

// POST - اذخوع دءاشن
export async function POST(request: NextRequest) {
  try {
    const user = await getCurrentUser(request)
    if (!user) {
      return NextResponse.json({ error: 'لوخدلا لجسم ريغ' }, { status: 401 })
    }

    if (!user.familyId) {
      return NextResponse.json({ error: 'ةلائاع يف وضع تسجل تنأ' }, { status: 400 })
    }

    // ؤوعدلا ؤيحالص نم ققحتلا
    if (user.familyRole !== 'owner' && user.familyRole !== 'admin') {
      return NextResponse.json({ error: 'ةوعدلا ؤيحالص لئدل سيل' }, { status: 403 })
    }

    const { email, role = 'member' } = await request.json()

    if (!email?.trim()) {
      return NextResponse.json({ error: 'بولطم ينورتكللا ديربلا' }, { status: 400 })
    }

    // ؤطشن ؤقباس ؤوعد دوجو مدع نم ققحتلا
    const existingInvitation = await prisma.familyInvitation.findFirst({
      where: {
        familyId: user.familyId,
        email: email.toLowerCase(),
        status: 'pending',
        expiresAt: { gt: new Date() }
      }
    })

    if (existingInvitation) {
      return NextResponse.json({
        error: 'لغفلاب ديربلا اذهل ؤطشن ؤوعد دجوي',
        existingCode: existingInvitation.code
      }, { status: 400 })
    }

    // لغفلاب مدخستسملا مامصنا مدع نم ققحتلا
    const existingMember = await prisma.user.findUnique({
      where: { email: email.toLowerCase() }
    })

    if (existingMember?.familyId === user.familyId) {
      return NextResponse.json({ error: 'ةلائاعلا يف لغفلاب وضع مدخستسملا اذه' }, { status: 400 })
    }

    // ؤوعدلا زمرءاشن
    const code = randomBytes(6).toString('hex').toUpperCase()
    const expiresAt = new Date()
    expiresAt.setDate(expiresAt.getDate() + 7) // عوبسأ ؤدمل ؤحلاص
  }
}
```

```

const invitation = await prisma.familyInvitation.create({
  data: {
    code,
    email: email.toLowerCase(),
    role,
    familyId: user.familyId,
    invitedBy: user.id,
    expiresAt
  }
})

// لئلا نلغ لولع لولع لولع لولع
const family = await prisma.family.findUnique({
  where: { id: user.familyId }
})

return NextResponse.json({
  success: true,
  invitation,
  shareLink: `${process.env.NEXT_PUBLIC_APP_URL || ''}/?invite=${code}`,
  familyName: family?.name
})
} catch (error) {
  console.error('Create invitation error:', error)
  return NextResponse.json({ error: 'أطخ دح' }, { status: 500 })
}
}

// GET - زمر نم ققح دل
export async function GET(request: NextRequest) {
  try {
    const { searchParams } = new URL(request.url)
    const code = searchParams.get('code')

    if (!code) {
      return NextResponse.json({ error: 'بولطم ةوع دل زمر' }, { status: 400 })
    }

    const invitation = await prisma.familyInvitation.findFirst({
      where: {
        code: code.toUpperCase(),
        status: 'pending',
        expiresAt: { gt: new Date() }
      },
      include: {
        family: {
          select: { id: true, name: true }
        },
        inviter: {
          select: { name: true, avatar: true }
        }
      }
    })

    if (!invitation) {
      return NextResponse.json({
        valid: false,
        error: 'ي هتتم وأ حل اص ريغ ةوع دل زمر',
        }, { status: 404 })
    }

    return NextResponse.json({
      valid: true,
      invitation: {
        code: invitation.code,
        email: invitation.email,
        role: invitation.role,
        familyName: invitation.family.name,
        inviterName: invitation.inviter.name,
        inviterAvatar: invitation.inviter.avatar
      }
    })
  } catch (error) {
    console.error('Verify invitation error:', error)
    return NextResponse.json({ error: 'أطخ دح' }, { status: 500 })
  }
}

```

17. src/app/api/scan-receipt/route.ts

```
□ /home/z/my-project/src/app/api/scan-receipt/route.ts
```

```
import ZAI from 'z-ai-web-dev-sdk';
import { NextRequest, NextResponse } from 'next/server';

// الجاهل المدخسمال ىلع لومحل
async function getCurrentUser(request: NextRequest) {
  try {
    const { prisma } = await import('@lib/prisma');

    const sessionToken = request.cookies.get('session-token')?.value
    if (!sessionToken) return null

    const session = await prisma.session.findFirst({
      where: {
        id: sessionToken,
        expiresAt: { gt: new Date() }
      },
      include: { user: true }
    })

    return session?.user || null
  } catch (error) {
    console.error('Auth error:', error)
    return null
  }
}

// ىفصارتفالا تافىنصتالا
const CATEGORIES = [
  { id: 'dairy', name: 'نابلأ', keywords: ['بىلح', 'نبل', 'نبح', 'يدابز', 'بور', 'طشق', 'دبزر', 'يريك', 'زوتكال', 'milk', 'نبح'] },
  { id: 'meat', name: 'موحل', keywords: ['محل', 'مةحل', 'رقب', 'فورخ', 'بابك', 'ةفك', 'مورفم', 'كيتس', 'همحل', 'ةمحل'] },
  { id: 'poultry', name: 'نجاود', keywords: ['اجاد', 'خارف', 'خرف', 'جورف', 'كيد', 'يمور', 'طب', 'زو', 'رودص', 'دخف', ''] },
  { id: 'seafood', name: 'كاسا', keywords: ['كس', 'نايبر', 'نايبور', 'يربم', 'ةنوت', 'نوملاس', 'نيدرس', ''] },
  { id: 'bakery', name: 'تازوبخم', keywords: ['زب', 'نومص', 'نوساورك', 'تسوت', 'ونيف', 'كعك', 'رطف', 'ناساورك', ''] },
  { id: 'vegetables', name: 'رامخ', keywords: ['مطامط', 'لمب', 'موث', 'سباطب', 'رزح', 'رايخ', 'لفلف', 'سج', 'ناجنذاب', 'اسوك', 'ب'] },
  { id: 'fruits', name: 'هكاوف', keywords: ['خافت', 'زوم', 'لاقترب', 'نوميل', 'بنع', 'نامر', 'لوارف', 'وجنام', 'يويك', 'رمت', ''] },
  { id: 'frozen', name: 'تادمجم', keywords: ['دمجم', 'جلثم', 'ميركسي', 'رزيرف', 'سويبس', 'ميركسي', ''] },
  { id: 'legumes', name: 'تايالوقب', keywords: ['لوق', 'سدغ', 'صمخ', 'ايبول', 'الزاب', ''] },
  { id: 'spices', name: 'تاراهبو لباوت', keywords: ['لباوت', 'تاراهب', 'جلم', 'ركس', 'لفلف', 'مكرك', 'لفرق', 'نارفعز', 'له', 'مك', 'لوكوش', 'اليتون', 'ةوالقب', 'ةفانك', 'ةوالح', 'تيوكسب', 'ك', 'ةتالوكوش', 'تايولح', ''] },
  { id: 'sweets', name: 'تايولح', keywords: ['تايولح', 'ريصع', 'بورشم', ''] },
  { id: 'drinks', name: 'تابورشم', keywords: ['تايولح', 'ريصع', 'بورشم', ''] },
  { id: 'snacks', name: 'سكانس', keywords: ['سكانس', 'سبيش', 'تارسكم', 'راشف', 'نروكبوب', ''] },
  { id: 'cleaning', name: 'تافطنم', keywords: ['فطنم', 'نوباص', 'قوحسم', 'رولك', 'رهطم', 'لوتيد', 'يريف', 'ليسيرب', 'ديات', ''] },
  { id: 'personal', name: 'ةيصخس ةيانع', keywords: ['وبماش', 'ميرك', 'نشول', 'نوجعم', 'اشرف', 'ليرم', 'تاضافح', 'زربماب', ''] },
  { id: 'kitchen', name: 'تاوداو خطم', keywords: ['ردق', 'ةالقم', 'مقط', 'نحص', 'بوك', 'ةقعلم', 'نيكس', ''] },
  { id: 'other', name: 'يرخأ', keywords: [] }
];

// جتنملا فينصت
function classifyProduct(productName: string): string {
  const nameLower = productName.toLowerCase();
  for (const cat of CATEGORIES) {
    if (cat.keywords.some(keyword => nameLower.includes(keyword.toLowerCase()))) {
      return cat.id;
    }
  }
  return 'other';
}

// منل نم JSON جارختسال ةلاد
function extractJSON(text: string): any {
  // رخآو لوا نع ثحبلا
  const firstBrace = text.indexOf('{');
  const lastBrace = text.lastIndexOf('}');

  if (firstBrace === -1 || lastBrace === -1 || firstBrace > lastBrace) {
    return null;
  }

  const jsonStr = text.substring(firstBrace, lastBrace + 1);

  try {
    return JSON.parse(jsonStr);
  } catch {
  }
}
```

```

    return null;
  }
}

export async function POST(request: NextRequest) {
  console.log('=== Scan Receipt API Called ===')

  try {
    // مدخلات المستخدم من قوقل
    const user = await getCurrentUser(request);
    console.log('User:', user ? user.email : 'Not logged in')

    if (!user) {
      return NextResponse.json({
        success: false,
        error: 'الرجاء تسجيل الدخول ليستخدم',
      }, { status: 401 });
    }

    // تاني بيل اءارق
    let body;
    try {
      body = await request.json();
    } catch (e) {
      console.error('Failed to parse request body')
      return NextResponse.json({
        success: false,
        error: 'البيانات اءارقت في ملف',
      }, { status: 400 });
    }

    const imageBase64 = body.image || body.imageBase64;

    if (!imageBase64) {
      return NextResponse.json({
        success: false,
        error: 'مطلوب صورة',
      }, { status: 400 });
    }

    // صورة طبارة زينة
    let imageUrl: string;
    if (imageBase64.startsWith('data:')) {
      imageUrl = imageBase64;
    } else {
      imageUrl = `data:image/jpeg;base64,${imageBase64}`;
    }

    console.log('Image size:', imageUrl.length, 'characters')

    // SDK
    console.log('Initializing ZAI SDK...')
    const zai = await ZAI.create();
    console.log('ZAI SDK initialized')

    // صورة ليحل
    console.log('Calling Vision API...')
    const completion = await zai.chat.completions.createVision({
      messages: [
        {
          role: 'user',
          content: [
            {
              type: 'text',
              text: 'رجاء اءاركو كرام ورسلا رءاوف ليحل و ءارق في رءاء تناء'
            }
          ]
        }
      ]
    })
  }
}

```

مهم:

1. ءءاء ءروءاءلا أرقا
2. ءروءاءلا في روكءم ءءنم لك ءءءءا
3. ءءء ءءنم لك:
 - بوءءم وه امك طبءلاب ءءنملا مسا
 - (طءق مقرر) رءءلا
 - (1 اءنأ صءءفا ءروءم نء مل اءا) ءمءلا

فيءاملا مالء في أوءب طءق JSON قءسءءلا اءء ءءءءءلا ءءراء

```

{
  "store": "رءءءلا مسا",
  "date": "ءءراءءلا",
  "items": [

```

```

    {
      "name": "حبتنملا مسلا",
      "price": 15.50,
      "quantity": 1
    }
  ],
  "total": 150.00
}

ةروئا فلأ هذه للآ
  },
  {
    type: 'image_url',
    image_url: {
      url: imageUrl
    }
  }
]
}
],
thinking: { type: 'disabled' }
});

const responseText = completion.choices[0]?.message?.content || '';
console.log('AI Response length:', responseText.length)
console.log('AI Response preview:', responseText.substring(0, 500))

// درلأ نم JSON آ ارآسا
const receiptData = extractJSON(responseText);

if (!receiptData) {
  console.error('No valid JSON found in response')
  return NextResponse.json({
    success: false,
    error: 'آصوأ ةروص برآ .ةروئا فلأ ةءارق نم نآمتن مل',
    rawResponse: responseText.substring(0, 1000)
  }, { status: 400 });
}

// آاآتنملا دوآو نم ققآآلأ
if (!receiptData.items || !Array.isArray(receiptData.items) || receiptData.items.length === 0) {
  console.error('No items found in receipt')
  return NextResponse.json({
    success: false,
    error: 'ةروصلأ آوصو نم دكأت .ةروئا فلأ ف آاآتنم دآن مل',
    rawResponse: responseText.substring(0, 1000)
  }, { status: 400 });
}

// آاآتنملا ففطنآو فففصت
const itemsWithCategory = receiptData.items
  .filter((item: any) => item && item.name && String(item.name).trim() !== '')
  .map((item: any) => ({
    name: String(item.name || '').trim(),
    price: Number(item.price) || 0,
    quantity: Number(item.quantity) || 1,
    category: classifyProduct(String(item.name || ''))
  }));

if (itemsWithCategory.length === 0) {
  return NextResponse.json({
    success: false,
    error: 'ةروئا فلأ نم ةآلأص آاآتنم آارآسا نم نآمتن مل',
    rawResponse: responseText.substring(0, 1000)
  }, { status: 400 });
}

console.log('Found', itemsWithCategory.length, 'items')

return NextResponse.json({
  success: true,
  store: receiptData.store || '',
  date: receiptData.date || '',
  items: itemsWithCategory,
  total: Number(receiptData.total) || 0
});
} catch (error: any) {
  console.error('=== Receipt scan error ===')
}

```

```
console.error('Error name:', error.name)
console.error('Error message:', error.message)

return NextResponse.json({
  success: false,
  error: 'فوریہ ریغ اطح' || (error.message || 'اطح شدح'),
}, { status: 500 });
}
}

// حومسمل request body لال مجج ەدایز
export const config = {
  api: {
    bodyParser: {
      sizeLimit: '10mb',
    },
  },
},
}
```


18. public/manifest.json

□ /home/z/my-project/public/manifest.json

```
{
  "name": "موقع - إلكتروني",
  "short_name": "موقع",
  "description": "موقع إلكتروني يوفر معلومات عن المنتجات والخدمات",
  "start_url": "/",
  "display": "standalone",
  "background_color": "#0c1222",
  "theme_color": "#10b981",
  "orientation": "portrait-primary",
  "scope": "/",
  "lang": "ar",
  "dir": "rtl",
  "icons": [
    {
      "src": "/icons/icon-72x72.png",
      "sizes": "72x72",
      "type": "image/png",
      "purpose": "maskable any"
    },
    {
      "src": "/icons/icon-96x96.png",
      "sizes": "96x96",
      "type": "image/png",
      "purpose": "maskable any"
    },
    {
      "src": "/icons/icon-128x128.png",
      "sizes": "128x128",
      "type": "image/png",
      "purpose": "maskable any"
    },
    {
      "src": "/icons/icon-144x144.png",
      "sizes": "144x144",
      "type": "image/png",
      "purpose": "maskable any"
    },
    {
      "src": "/icons/icon-152x152.png",
      "sizes": "152x152",
      "type": "image/png",
      "purpose": "maskable any"
    },
    {
      "src": "/icons/icon-192x192.png",
      "sizes": "192x192",
      "type": "image/png",
      "purpose": "maskable any"
    },
    {
      "src": "/icons/icon-384x384.png",
      "sizes": "384x384",
      "type": "image/png",
      "purpose": "maskable any"
    },
    {
      "src": "/icons/icon-512x512.png",
      "sizes": "512x512",
      "type": "image/png",
      "purpose": "maskable any"
    }
  ],
  "categories": ["shopping", "finance", "lifestyle"],
  "screenshots": [],
  "shortcuts": [
    {
      "name": "إضافة منتج",
      "short_name": "إضافة",
      "description": "إضافة منتج جديد",
      "url": "?action=add",
      "icons": [{ "src": "/icons/icon-96x96.png", "sizes": "96x96" }]
    }
  ]
}
```

```
"related_applications": [],  
"prefer_related_applications": false  
}
```

19. public/sw.js

```
□ /home/z/my-project/public/sw.js
```

```
const CACHE_NAME = 'home-items-v1';
const STATIC_CACHE = 'static-v1';
const DYNAMIC_CACHE = 'dynamic-v1';

// اتقۇم اەن يىزىخت مەن سىس يىتلە تاقلەملا
const STATIC_FILES = [
  '/',
  '/manifest.json',
  '/offline.html'
];

// تەييارلا Service Worker
self.addEventListener('install', (event) => {
  console.log('[SW] Installing Service Worker...');

  event.waitUntil(
    caches.open(STATIC_CACHE)
      .then((cache) => {
        console.log('[SW] Caching static files');
        return cache.addAll(STATIC_FILES);
      })
      .then(() => {
        console.log('[SW] Service Worker installed');
        return self.skipWaiting();
      })
      .catch((error) => {
        console.error('[SW] Failed to cache:', error);
      })
  );
});

// لىغەت Service Worker
self.addEventListener('activate', (event) => {
  console.log('[SW] Activating Service Worker...');

  event.waitUntil(
    caches.keys()
      .then((cacheNames) => {
        return Promise.all(
          cacheNames
            .filter((name) => name !== STATIC_CACHE && name !== DYNAMIC_CACHE)
            .map((name) => {
              console.log('[SW] Deleting old cache:', name);
              return caches.delete(name);
            })
        );
      })
      .then(() => {
        console.log('[SW] Service Worker activated');
        return self.clients.claim();
      })
  );
});

// تاقۇملا نىزىختەلە جەجىتەرتسا
self.addEventListener('fetch', (event) => {
  const { request } = event;
  const url = new URL(request.url);

  // Chrome extension و non-GET تابلەت
  if (url.protocol === 'chrome-extension:' || request.method !== 'GET') {
    return;
  }

  // Network First لىل API جەجىتەرتسا
  if (url.pathname.startsWith('/api/')) {
    event.respondWith(networkFirst(request));
    return;
  }

  // تابلەتلە تاقلەملا Cache First جەجىتەرتسا
  event.respondWith(cacheFirst(request));
});
```

```

// Network First - يكيما نيدلا يوتحملل
async function networkFirst(request) {
  try {
    const networkResponse = await fetch(request);

    if (networkResponse.ok) {
      const cache = await caches.open(DYNAMIC_CACHE);
      cache.put(request, networkResponse.clone());
    }

    return networkResponse;
  } catch (error) {
    const cachedResponse = await caches.match(request);

    if (cachedResponse) {
      return cachedResponse;
    }

    return new Response(
      JSON.stringify({ error: 'تنترن لابل لصتم ريغ تنأ', offline: true }),
      {
        status: 503,
        headers: { 'Content-Type': 'application/json' }
      }
    );
  }
}

// Cache First - تافلملل
async function cacheFirst(request) {
  const cachedResponse = await caches.match(request);

  if (cachedResponse) {
    // يف شاكللا شيدحت
    fetch(request)
      .then((networkResponse) => {
        if (networkResponse.ok) {
          caches.open(STATIC_CACHE)
            .then((cache) => cache.put(request, networkResponse));
        }
      })
      .catch(() => {});

    return cachedResponse;
  }

  try {
    const networkResponse = await fetch(request);

    if (networkResponse.ok) {
      const cache = await caches.open(DYNAMIC_CACHE);
      cache.put(request, networkResponse.clone());
    }

    return networkResponse;
  } catch (error) {
    // لقنتلل offline ةحفص
    if (request.mode === 'navigate') {
      return caches.match('/offline.html');
    }

    return new Response('Offline', { status: 503 });
  }
}

// قيبطتل نم لئاسرلا لابقتسا
self.addEventListener('message', (event) => {
  if (event.data === 'skipWaiting') {
    self.skipWaiting();
  }

  if (event.data.type === 'SYNC_DATA') {
    // لاصتال دنع تانايبلا ةنمازم
    syncData();
  }
});

// تانايبلا ةنمازم
async function syncData() {

```

```

try {
  const clients = await self.clients.matchAll();
  clients.forEach((client) => {
    client.postMessage({ type: 'SYNC_COMPLETE' });
  });
} catch (error) {
  console.error('[SW] Sync failed:', error);
}
}

// Push Notifications
self.addEventListener('push', (event) => {
  const options = {
    body: event.data?.text() || 'ديدج شي دجت!',
    icon: '/icon-192x192.png',
    badge: '/icon-72x72.png',
    vibrate: [100, 50, 100],
    data: {
      dateOfArrival: Date.now(),
      primaryKey: 1
    },
  },
  actions: [
    { action: 'open', title: 'قريب طتل حتف' },
    { action: 'close', title: 'قالغ' }
  ]
};

  event.waitUntil(
    self.registration.showNotification('يضا قم', options)
  );
});

// راعش الال ىلع رقنل
self.addEventListener('notificationclick', (event) => {
  event.notification.close();

  if (event.action === 'open' || !event.action) {
    event.waitUntil(
      clients.openWindow('/')
    );
  }
});

console.log('[SW] Service Worker loaded');

```

20. src/app/page.tsx

```
/home/z/my-project/src/app/page.tsx
```

```
import { PrismaClient } from '@prisma/client'
import { NextResponse } from 'next'
import { headers } from 'next/headers'

const prisma = new PrismaClient()

export async function GET() {
  const headersList = headers()
  const token = headersList.get('token')

  if (!token) {
    return NextResponse.json({ error: 'Token is required' }, { status: 401 })
  }

  const user = await prisma.user.findUnique({
    where: { token },
  })

  if (!user) {
    return NextResponse.json({ error: 'Invalid token' }, { status: 401 })
  }

  return NextResponse.json({ user })
}
```

Database Setup

1. Create a new database instance (e.g., PostgreSQL)
2. Create a new database user and grant privileges
3. Create a new .env file in the root of the project with the following content: DATABASE_URL="file:./dev.db"
4. Run the following commands to set up the database:

```
npm install
npx prisma generate
npx prisma db push
npm run dev
```
5. The application is now running on http://localhost:3000